Machine learning

Toghrul Karimov
Max Planck Institute for Software Systems

Last updated in 04.2026

Contents

1 Introduction

In this course, we work in the setting of *supervised learning*. Here we have variables (i.e., columns) $X_1, \dots, X_m, Y$, where X_i are the independent variables and Y is the dependent variable. For example:

- $m = 1$, $X_1 > 0$ is the height of a person, and $Y \in \{\text{male}, \text{female}\}$ is their gender.
- $m = 2$, $X_1 \in \{\text{high school}, \text{bachelor's}, \text{master's}, \text{phd}\}$ is the education level of an employee, $X_2 \geq 0$ is the experience in years, and $Y > 0$ is their salary.
- $m = 256$, $X_1, \dots, X_{256} \in [0, 1]$ are the (scaled) values of pixels of a 16×16 image in greyscale, and $Y \in \{\text{cat}, \text{dog}, \text{horse}\}$ is the class of the animal in the picture.

We suppose that we have n observations: x_i is a (row) vector of length m, where $x_{i,j}$ is the value of X_j for the ith observation, and y_i is the value of Y for the ith observation. We arrange the data into a matrix form as

$$\mathbf{X} = \begin{bmatrix} \leftarrow x_1 \rightarrow \\ \vdots \\ \leftarrow x_n \rightarrow \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} y_1 \\ \vdots \\ y_n \end{bmatrix}.$$

Thus $\mathbf{X}$ is a 2D array of dimensions $n \times m$, and $\mathbf{Y}$ is a 1D array of dimension n. We also assume that we have *test data* of the form $\mathbf{X}_{test}, \mathbf{y}_{test}$ where $\mathbf{X}_{test}$ has dimensions $n' \times m$ for some $n' > 0$, and $\mathbf{y}_{test}$ has dimension n'.

We have two general objectives.

1. Understand the relationship between Y (called the target/dependent variable) and $X_1, \ldots, X_m$ (called predictor/independent variables);
2. Given fresh $X_1, \ldots, X_m$, predict Y.

Our problem is a *classification* problem if Y is *categorical* (i.e., can take finitely many possible values), and a *regression* problem if Y is numeric.

2 Linear regression

Suppose $X_1, \ldots, X_m, Y$ are all numeric. The simplest way to model Y in terms of $X_1, \ldots, X_m$ is using a linear function:

$$Y = a_1 X_1 + \cdots + a_m X_m + b$$

where $a_1, \ldots, a_m, b$ are real-valued coefficients to be determined. How do we find good values for the coefficients? In the *least squares* approach, we minimise the *error (also called a loss) function*

$$\mathrm{MSE}(\widehat{\mathbf{y}}, \mathbf{y})$$

where MSE is the *mean-squared error* defined as

$$\mathrm{MSE}(\mathbf{u}, \mathbf{v}) = \frac{1}{n} \sum_{i=1}^{n} (u_i - v_i)^2$$

for two vectors $\mathbf{u} = (u_1, \ldots, u_n), \mathbf{v} = (v_1, \ldots, v_n)$ and $\widehat{\mathbf{y}}$ is the vector of predictions:

$$\widehat{\mathbf{y}} = \mathbf{X} \cdot [a_1\ a_2\ \cdots\ a_m]^\top + [b\ b\ \cdots\ b]^\top = \begin{bmatrix} a_1 x_{1,1} + \cdots + a_m x_{1,m} + b \\ \vdots \\ a_1 x_{n,1} + \cdots + a_m x_{n,m} + b \end{bmatrix}.$$

Observe that $\mathrm{MSE}(\widehat{\mathbf{y}}, \mathbf{y})$ is a quadratic function of $a_1, \ldots, a_m, b$. Under some mild conditions ($n \geq m$ and $\mathbf{X}$ has linearly independent columns), the coefficients minimising $\mathrm{MSE}(\widehat{\mathbf{y}}, \mathbf{y})$ are unique. They can be found using partial derivatives, gradient descent, or the formula

$$[b\, a_1\, a_2\ \cdots\ a_m]\top = (\mathbf{Z}^\top \mathbf{Z})^{-1} \mathbf{Z}^\top \mathbf{y}$$

where

$$\mathbf{Z} = \begin{bmatrix} 1 & x_{1,1} & x_{1,2} & \cdots & x_{1,m} \\ & & \vdots & & \\ 1 & x_{n,1} & x_{n,2} & \cdots & x_{n,m} \end{bmatrix}$$

is the $n \times (m+1)$ *design matrix*. Once we have have found the coefficients, we can evaluate the performance of our model on unseen data by

1. computing the vector of predictions $\widehat{\mathbf{y}}_{test} = \mathbf{X}_{test} \cdot [a_1\ a_2\ \cdots\ a_m]^\top + [b\ b\ \cdots\ b]^\top$ and
2. computing the error $\mathrm{MSE}(\widehat{\mathbf{y}}_{test}, \mathbf{y}_{test})$.

In linear regression it is recommended to scale (at least) the independent variables $X_1, \ldots, X_m$, to obtain new variables that have mean 0 and standard deviation 1. Let μ_i, σ_i be the sample mean and sample standard deviation of X_i, both in $\mathbf{X}$. We then perform the transformation $\widetilde{X}_i = \frac{X_i - \mu_i}{\sigma_i}$, and learn a model

$$Y = a_1 \widetilde{X}_1 + \cdots + a_m \widetilde{X}_m + b.$$

Here $\widetilde{X}_1, \ldots, \widetilde{X}_m$ are all on the same scale: they have (sample) mean 0 and (sample) standard deviation 1. Thus it is possible to interpret the coefficients.

1. If $|a_i| \gg |a_j|$, it means that X_i has a larger influence on Y than X_j.
2. The value of b is the value of Y for an "average" data point, i.e. the point $X_1 = \cdots = X_m = 0$.

It is very important that when you scale the training data, you perform exactly the same mathematical steps on the test data. Thus if we evaluate our learned model on the test data by first transforming $\mathbf{X}_{test}$ using the means and standard deviations μ_i, σ_i that we learned on the training data. Observe that in this case, after the transformation the columns will not necessarily have mean 0 and standard deviation 1.

Linear regression is sensitive to *outliers*. When the distribution of Y has a large tail (e.g., when modelling salaries, where a small number of people earn orders of magnitude more than the average), the *homoscedasticity* assumption of linear regression is violated. In this case, a common approach is to model $\log(Y)$ linearly in terms of $X_1, \ldots, X_m$: taking the log "compresses" the outliers.

It is possible to apply linear regression to problems where some X_i are categorical. There are several approaches to this. Suppose $X_i \in \{v_1, \ldots, v_k\}$.

1. Drop the categorical variables and only keep numeric ones. This loses potentially very useful information.

2. Replace $v_1, \ldots, v_k$ with some numbers, e.g. $1, \ldots, k$. This is appropriate only when the categories $v_1, \ldots, v_k$ are inherently linearly ordered, e.g. when X_i stores education level of a person. Otherwise, such a transformation imposes an artificial order on the values of X_i, and is methodologically incorrect in the context of linear regression.

3. One-hot encoding. In this case, we replace X_i with k new columns $X_{i,1}, \ldots, X_{i,k}$, and translate $X_i = v_l$ following
$$X_{i,j} \in \{0, 1\} \text{ for all } j \text{ and } X_{i,j} = 1 \Leftrightarrow j = l.$$
Observe that if we know the values of $k-1$ columns, we can infer the value of the remaining column. Hence the columns contain redundancy (in particular, they are *multi-collinear*), which is bad for linear regression. The correct approach is thus *one-hot encoding with drop one*, where we drop one of the columns: in the end, we replace X_i with $k-1$ columns, not k.

3 Polynomial regression, overfitting, and cross-validation

Now suppose $m = 1$, and $X \coloneqq X_1$ and Y are numeric. We will model $Y = p(X)$, where p is a polynomial that is to be determined. Polynomial regression is also applicable when $m > 1$: in this case, p is a multi-variate polynomial in m variables, and the model is $Y = p(X_1, \ldots, X_m)$.

For the purposes of this section, we assume that (X, Y) are generated (by the nature) as follows. First, X is sampled uniformly from the interval $[1, 10]$. Then Y is computed by first applying the cubic polynomial $f(x) = 0.105x^3 - 1.68x^2 + 7.86x - 5.29$ to X, and then adding a normally distributed random variable with mean 0 and variance 1. (In particular, we can generate as many data points from this distribution as we want.) Fig. 1 shows that graph of f and 50 randomly generated data points $(x_1, y_1), \ldots, (x_{50}, y_{50})$, which will be our training data.

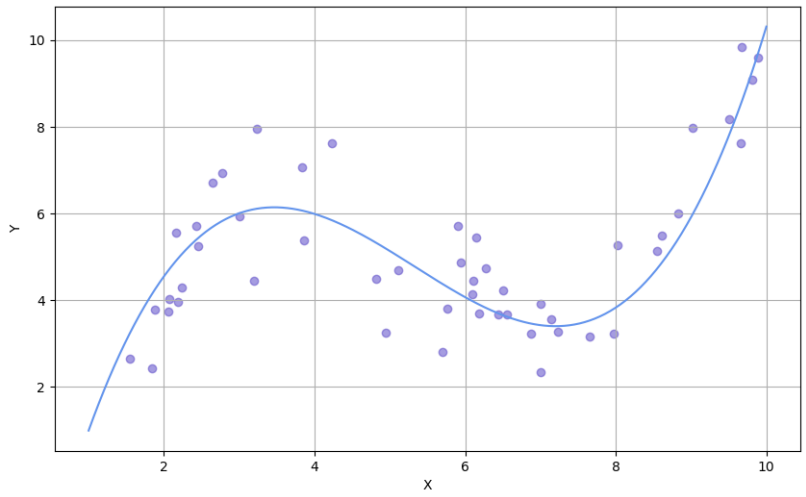

Figure 1: The function f and 50 points (x, y) satisfying $y = f(x) + \mathcal{N}(0, 1)$.

Of course, we only see the generated points and not the "true function" f. Which polynomial p should we choose? Firstly, if we have 50 points, under a reasonable assumption there exists a unique polynomial p of degree 49 such that the model $Y = p(X)$ has the (mean squared) error of 0 on our data. Hence we don't need to consider polynomials of degree 50 or higher. We can find, for each degree $d = 0, 1, \ldots, 49$, the polynomial p that fits our data best, i.e. minimises the error

$$\frac{1}{50}\sum_{i=1}^{50}(y_i - p(x_i))^2.$$

Fig. 2 shows the best polynomial (as far as fitting our dataset is concerned) for degrees $d = 0, \ldots, 11$.

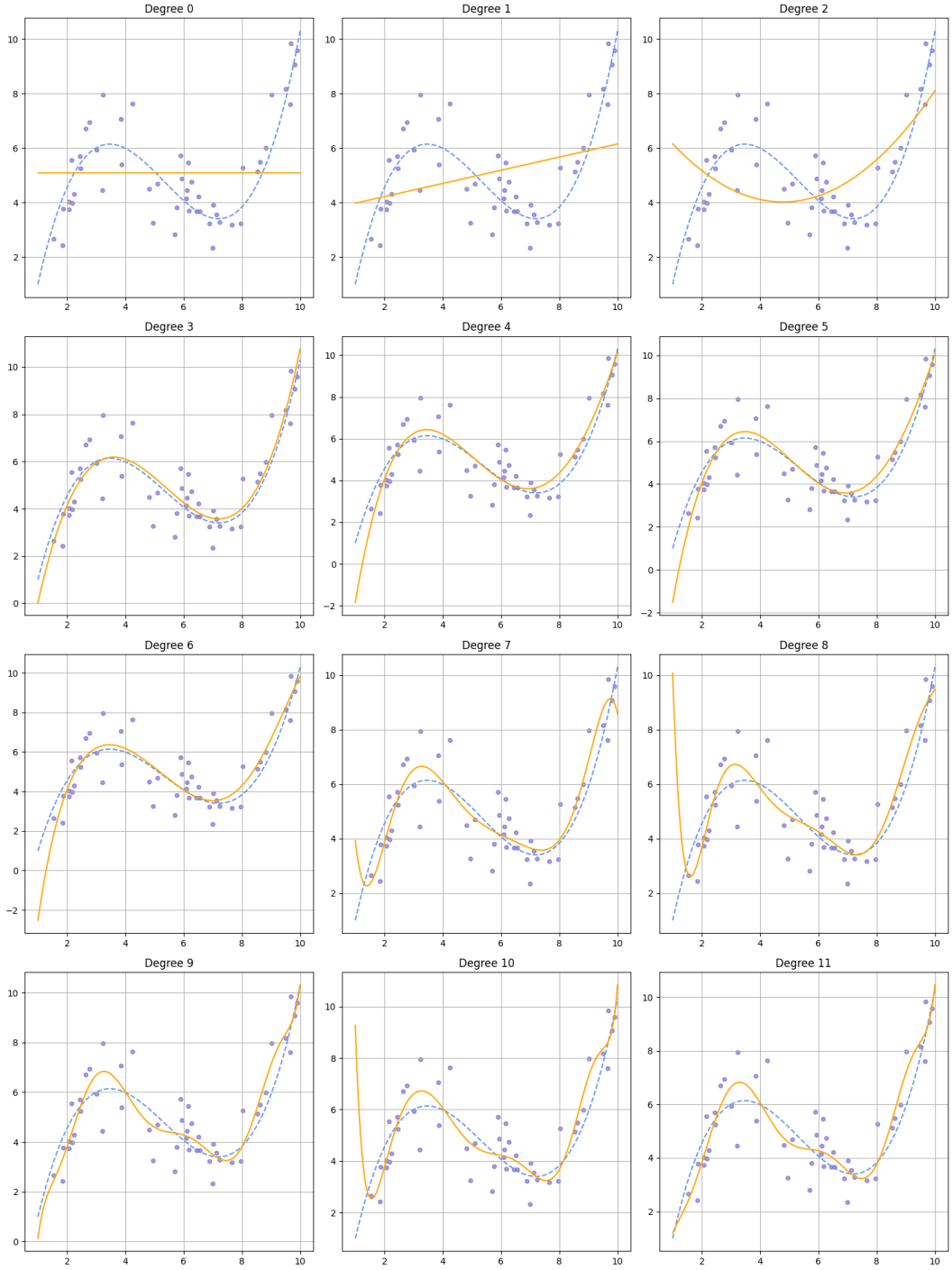

Figure 2: The best polynomial for each degree, with respect to our training set and according to the least squares criterion.

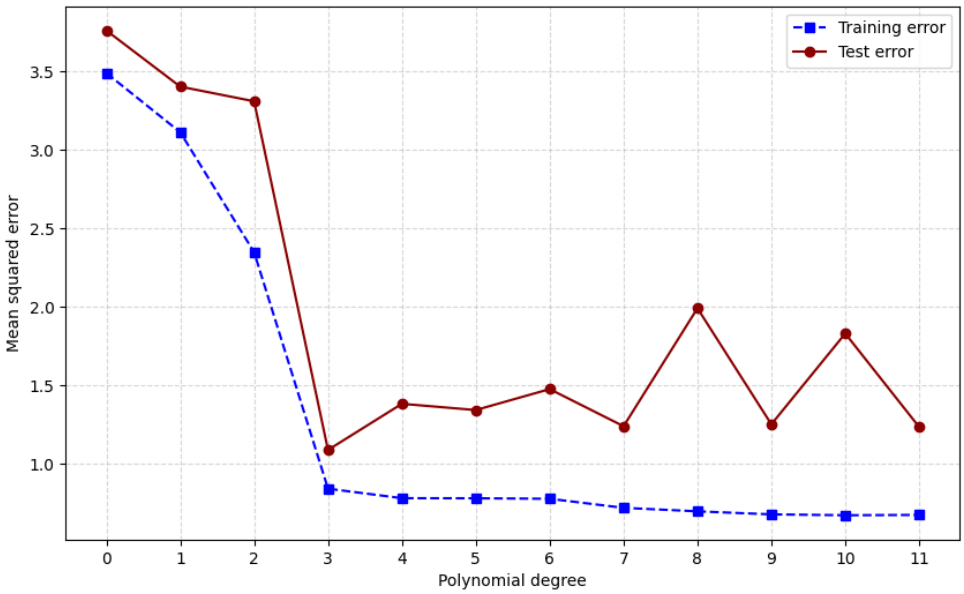

Figure 3: Performance of the best polynomial for each degree on the training data and unseen (test) data.

As we increase the degree, we see that the polynomials fit our training data better and better. This is also demonstrated in Fig. 3, blue line. However, the story is very different if generate large test data and evaluate the "best polynomial" at each degree on the test data (Fig. 3, red line). We see that the model $d = 3$ performs best with MSE of approximately 1; recall that $Y = f(X) + \mathcal{N}(0, 1)$. So even though allowing higher degrees should, in theory, make our models more powerful, in reality their performance decreases on unseen data.

We have the following well-known situation.

- The models with degree 0, 1, 2 are *underfitting*: they are unable to capture the true relationship between X and Y, and thus have high error on unseen data.
- The models with degree 4 and more are *overfitting*: they are learning the true relationship between X and Y, as well as *noise*, i.e. information that is specific to only our dataset and does not generalise to full data.

Unsurprisingly, the best choice is $d = 3$. However, purely numerically, we see that the performance of the best model with $d = 7$ is similarly good. Is there an argument for choosing $d = 7$? In these cases we employ *Occam's razor*: if two models have similar performance, then the simpler one should be preferred. In this case, it is the cubic one.

The degree of the polynomial p to be fitted is a *hyperparameter*: it controls how powerful our model will really be. Hyperparameters of a model family need to be *tuned* before the final model can be learned. One approach for this (that is especially good when we do not have access to copious amounts of data) is *k-fold cross-validation*. It works as follows.

We first choose k based on experience; typically $k = 5$ is admissible. We then (randomly) split our training data into k folds, i.e. k pieces $(\mathbf{X}_1, \mathbf{y}_1), \ldots, (\mathbf{X}_k, \mathbf{y}_k)$ of equal size. Each $\mathbf{X}_i, \mathbf{y}_i$ have dimensions (assuming n is divisible by k) $n/k \times m$ and $n/k \times 1$, respectively. We then select a finite set of possible values for our hyperparameters: in our case, this was $d \in \{0, 1, \ldots, 11\}$. For each possible combination of values of the hyperparameters, we compute a *cross-validation score* as follows. We fix the hyperparameter values for now. For each fold $(\mathbf{X}_i, \mathbf{y}_i)$, we train the best possible model with the fixed hyperparameter values on the remainder of the training data, and evaluate it on $(\mathbf{X}_i, \mathbf{y}_i)$. We then take the average of k scores. Fig. 4 shows the cross-validation scores for the degree $d \in \{0, 1, \ldots, 11\}$ on our training data of 50 points. Once we have these, we then pick the hyperparameter values by choosing the simplest model family whose performance is the best or close to the best: in this case, this does yield the degree 3.

At the end of cross-validation, we re-train our model on the full training data with the hyperparameter values that we found. In our case, this yields $p(x) = 0.113x^3 - 1.83x^2 + 8.80x - 7.05$.

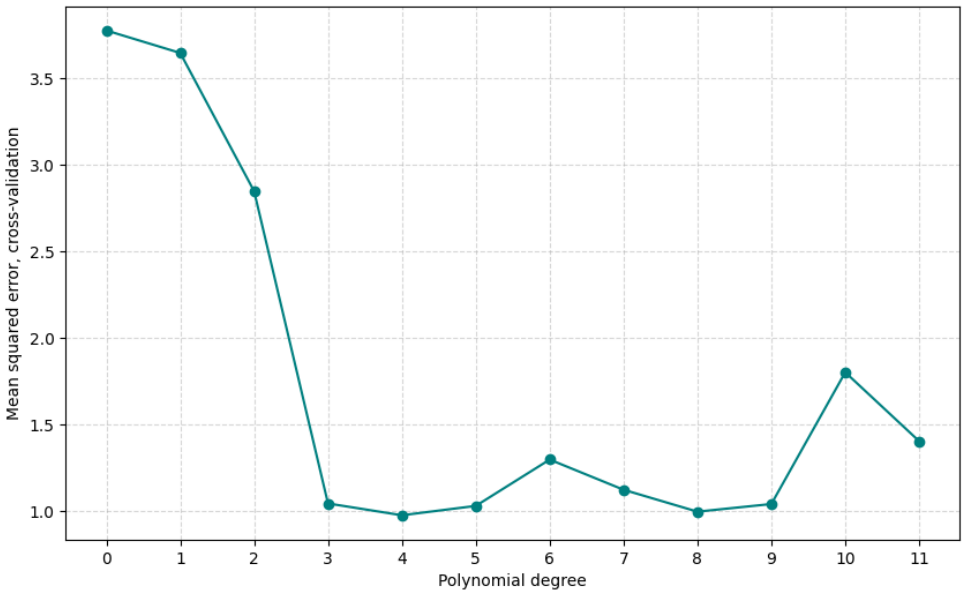

Figure 4: Cross validation scores for the degrees $d \in \{0, 1, \ldots, 11\}$.

Compare this with the "true function" $f(x) = 0.105x^3 - 1.68x^2 + 7.86x - 5.29$.

4 Lasso and ridge regression

A potential problem for linear regression is *multi-collinearity*, i.e. some columns of the data being (essentially) linearly dependent. Suppose, for example, we have a dataset about the salary of the employees with columns $X_1, \ldots, X_5, Y$ as follows.

- X_1 is the age in years.
- X_2 is the experience in years.
- $X_3, X_4, X_5 \in \{0, 1\}$.
- $X_3 = 1 \Leftrightarrow$ has an undergraduate degree.
- $X_4 = 1 \Leftrightarrow$ has a master's degree.
- $X_5 = 1 \Leftrightarrow$ has a PhD.
- Y is the monthly salary in euros.

Then we have the approximate linear relation

$$X_1 \approx 20 + 3X_3 + 2X_4 + 4X_5 + X_2.$$

Now suppose the "true" model of Y is

$$Y \approx 2000 + 200X_2 + 250X_3 + 150X_4 - 100X_5.$$

That is, the baseline pay is 2000, each year of experience adds 200, having an undergraduate degree adds 250, a master's degree adds 150, and because PhD's usually work in lower-paying jobs, having a PhD decreases your salary a little on average. But because of the aforementioned linear relation, we have many other valid models, e.g.

$$Y \approx 100X_1 + 100X_2 - 50X_3 - 50X_4.$$

It is impossible interpret such a model in a natural way. In summary, the problems of multi-collinearity in data are at least two-fold.

- It makes learning *unstable*. Slight changes in training data result in very different learned coefficients.
- It makes the end result less interpretable.

Lasso and ridge are forms of *regularised* linear regression which penalise the model for using large coefficients. In lasso, we minimise

$$\text{MSE}(\widehat{\mathbf{y}}, \mathbf{y}) + \lambda(|a_1| + \cdots + |a_m|)$$

and in ridge we minimise

$$\text{MSE}(\widehat{\mathbf{y}}, \mathbf{y}) + \lambda(a_1^2 + \cdots + a_m^2).$$

In both cases, λ is a positive hyper-parameter that needs to be tuned: typical values to try in cross-validation are powers of 2 or 10, e.g. $0.001, 0.01, 0.1, 1, 10, 100$ etc. When λ is small (i.e., much smaller that $1/n$: note that it is not averaged over the data points unlike MSE), little regularisation happens, and the model behaves similarly to the ordinary linear regression model. When λ is too large, all of the coefficients will be zero or close to zero.

Although they look similar, lasso and ridge behave differently. Lasso aggressively sets coefficients to zero whenever possible: in our example above, at least one of the coefficients of X_1 and X_2 is likely to be exactly zero. Lasso thus can be used for *feature selection*, i.e. determining which independent variables actually determine the value of Y and which are redundant. Ridge, on the other hand, shrinks the coefficients as much as possible, but generally does not set them to zero. In our example, it is likely to assign similar positive coefficients to X_1 and X_2. Ridge is used when we believe that all independent features do in fact contribute to Y.

5 Logistic regression

We now move on to classification problems. For now, suppose $X_1, \ldots, X_m$ are all numeric, and Y is binary, i.e. $Y \in \{0, 1\}$. In this and the following sections we will see various approaches to "turning linear regression into a classification algorithm".

A basic classification algorithm predicts $Y = 0$ or $Y = 1$. A more powerful algorithm would first estimate the probability of $Y = 0$ and $Y = 1$, and then predict the class (if necessary) by picking $Y = 1$ if and only if $\Pr(Y = 1) > 0.5$. Let

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

for $x \in \mathbb{R}$. Observe that σ is a bijection from $\mathbb{R}$ to $(0, 1)$. In logistic regression, we model

$$\Pr(Y = 1) = \sigma(a_1 x_1 + \cdots + a_m x_m + b).$$

Even though σ is non-linear, its *decision boundaries* are linear: it predicts $Y = 1$ on input $X_1 = x_1, \ldots, X_m = x_m$ if and only if $a_1 x_1 + \cdots + a_m x_m + b > 0$.

We can evaluate the performance of a classification algorithm using

$$\text{Accuracy}(\widehat{\mathbf{y}}, \mathbf{y}) \coloneqq \frac{\#\text{ of } i \text{ such that } \widehat{y}_i = y_i}{n}$$

or a variation thereof, where n is the number of samples, $\widehat{\mathbf{y}} = (\widehat{y}_1, \ldots, \widehat{y}_n)$ is the vector of predicted class labels and $\mathbf{y} = (y_1, \ldots, y_n)$ is the vector of true labels. That is, we just measure the proportion of correct classifications. (We can adjust this to accommodate the relative *frequencies* of different classes: see, e.g., `https://en.wikipedia.org/wiki/Evaluation_of_binary_classifiers`.) However, when our model predicts probabilities, there is a more nuanced way of evaluating performance that is motivated by information theory. Now suppose $\widehat{\mathbf{y}}$ is the vector of probabilities of $Y = 1$, and $\mathbf{y}$ is the vector of true labels. Then we minimise the average *cross-entropy loss*

$$\frac{1}{n} \sum_{i=1}^{n} -(y_i \log \widehat{y}_i + (1 - y_i) \log(1 - \widehat{y}_i)).$$

This rewards confident correct predictions and penalises confident incorrect ones.

What if $Y \in \{0, \ldots, \ell\}$ where $\ell \geq 2$, i.e. we have more than two classes to predict? The standard way to do this is *multinomial logistic regression*. (We will discuss two more generic ways below.) Here we have coefficients $a_i^{(j)}, b^{(j)}$ for $0 \leq j < \ell$ and $1 \leq i \leq m$. The model for Y in terms of $X_1, \ldots, X_m$ is as follows. For $0 \leq j < \ell$, let $Z_j = a_1^{(j)} X_1 + \cdots + a_m^{(j)} X_m + b^{(j)}$. Then

$$\Pr(Y = 0), \ldots, \Pr(Y = \ell - 1) = \underbrace{\frac{e^{Z_0}}{e^{Z_0} + \cdots + e^{Z_{\ell-1}}}, \ldots, \frac{e^{Z_{\ell-1}}}{e^{Z_0} + \cdots + e^{Z_{\ell-1}}}}_{\mathrm{Softmax}(Z_0, \ldots, Z_{\ell-1})}.$$

The prediction for Y is then

$$\operatorname*{argmax}_j \frac{e^{Z_j}}{e^{Z_0} + \cdots + e^{Z_{\ell-1}}}.$$

A generic way to achieve classification with more than two labels is as follows: we train ℓ logistic classifiers, where the ith classifier predicts $\Pr(Y = i)$. We then select the class whose probability was the highest: this is known as the *one-vs-all* approach. In the *one-vs-one* approach, we train $\ell(\ell-1)/2$ pairs of classifiers, one for each pair of distinct labels. The classifier for the pair (j_1, j_2) of classes votes for one of the two. In the end, we aggregate the votes.

6 Support vector machines

We now discuss another linear approach to classification. In this section, we again assume that $X_1, \ldots, X_m$ are numeric and Y is binary. It will be convenient to take $Y \in \{1, -1\}$.

Suppose our training data is perfectly separable by a hyperplane: there exist $a_1, \ldots, a_m, b$ (and possibly many other combinations of coefficients) such that, for all $1 \leq i \leq n$,

$$y_i = 1 \Leftrightarrow a_1 x_{i,1} + \cdots + a_m x_{i,m} + b \geq 0.$$

Which coefficients should we take for our separating hyperplane? Is there any advantage to one choice over another? A support vector machine (SVM) tries to find the hyperplane that maximises the *margin*, which, for $a_1, \ldots, a_m$, is the width of the strip consisting of all hyperplanes with normal vector $(a_1, \ldots, a_m)$ that perfectly separate the two classes. The points belonging to either of the classes that constrain the margin are called the *support vectors*.

To learn the coefficients we proceed as follows. Firstly, observe that if the hyperplane defined by $a_1, \ldots, a_m, b$ perfectly separates our data, then so does the one defined by $ka_1, \ldots, ka_m, kb$ for any $k > 0$. Hence, controlling for scaling, it suffices to only consider the coefficients that satisfy

- $y_i(a_1 x_{i,1} + \cdots + a_m x_{i,m} + b) \geq 1$ for all $1 \leq i \leq n$, and
- $y_i(a_1 x_{i,1} + \cdots + a_m x_{i,m} + b) = 1$ for some i.

Among these hyperplanes, we try to maximise the margin, which happens to be equal to

$$\frac{2}{\sqrt{a_1^2 + \cdots + a_m^2}}.$$

Equivalently, we have the following optimisation problem: minimise $\frac{1}{2}(a_1^2 + \cdots + a_m^2)$ subject to $y_i(a_1 x_{i,1} + \cdots + a_m x_{i,m} + b) \geq 1$ for all i. This can be solved efficiently using algorithms from convex analysis.

What if data is not perfectly separable by a hyperplane? In this case, SVM tries to strike a balance between classifying as many points correctly as possible and maximising the margin. Formally, it minimises

$$\frac{1}{2}(a_1^2 + \cdots + a_m^2) + C \sum_{i=1}^{n} \max(0, 1 - y_i \widehat{y}_i)$$

where $\widehat{y}_i = a_1 x_{i,1} + \cdots + a_m x_{i,m} + b$ and $C > 0$ is a hyperparameter that needs to be tuned. The value $\max(0, 1 - y_i \widehat{y}_i)$ is called the *hinge loss*: it is

- 0 when $\widehat{y}_i$ has the correct sign and satisfies $|\widehat{y}_i| \geq 1$ (correct classification and outside the margin),

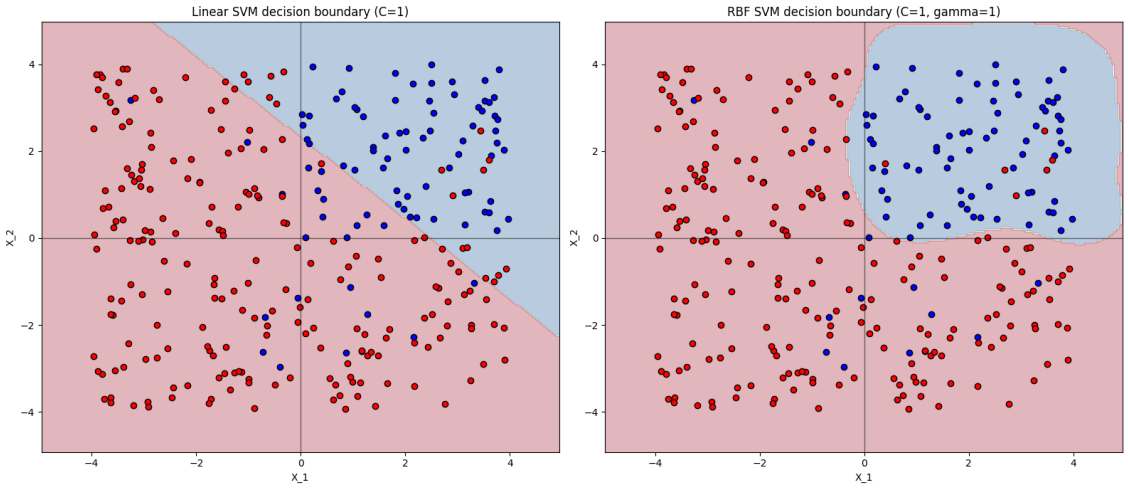

Figure 5: Decision boundaries for SVM with and without a kernel.

- between 0 and 1 when $\widehat{y}_i$ has the correct sign but $|\widehat{y}_i| < 1$ (correct classification but inside the margin), and
- greater than 1 if $\widehat{y}_i$ does not have the correct sign.

When C is large, the model tries hard to classify as many points correctly as possible. When C is small, the model cares less about incorrect classifications as long as they happen not too far away from the linear boundary.

SVMs can learn non-linear boundaries when augmented with *kernels*. In this case, the model operates as the usual SVM but in a higher-dimensional space of features (that are computed from the known m features). Fig. 5 depicts the decision boundaries of an SVM with *radial basis function* (RBF) as kernel (which has its own hyperparameter $\gamma > 0$ that needs to be tuned together with C). Observe that in this case data is mostly concentrated in the first quadrant with a small but non-trivial number of exceptions.

7 K-nearest neighbours

In k-nearest neighbours (kNN), the algorithm simply "memorises" the training data, and when presented with an unseen point x, makes its prediction based on the k points $x^{(1)}, \ldots, x^{(k)}$ in the training data that are nearest to x according to some metric, e.g. the Euclidean distance, the Manhattan distance etc. The value of k is a hyperparameter that needs to be tuned: we do not have any "usual" parameters in this case.

KNN can be used for both classification and regression. In classification, the algorithm looks at the labels of the k nearest points, and returns the label that occurs most frequently. If k is odd and Y is binary, this is guaranteed to return a unique prediction; otherwise we might have a tie, and need a tie-breaking mechanism. In regression, the algorithm simply return the average of Y-values of the k nearest neighbours.

In both classification and regression, in order to apply kNN correctly we first need to scale $X_1, \ldots, X_m$ to make sure that they are comparable: for example, we can perform the standard scaling

$$\widetilde{X}_i = \frac{X_i - \mu_i}{\sigma_i}$$

where μ_i is the mean of X_i and σ_i is its standard deviation. Then $\widetilde{X}_i$ will have mean 0 and standard deviation 1. **Important:** you must apply exactly the same transformation to the test data, i.e. use the values μ_i, σ_i computed from the training data. In particular, after scaling the mean and the standard deviation of the test data will not be exactly 0 and 1.

Let us look at an example that illustrates the importance of scaling. Suppose we have $m = 2$ and the data depicted in Fig 6. The distribution underlying the data is as follows. We have $\mathrm{P}(Y = 0) = \mathrm{P}(Y = 1)$. For the blue points (i.e., $Y = 0$), $X_1 \sim \mathcal{N}(-1, 1)$, and for the red points, $X_1 \sim \mathcal{N}(1, 1)$.

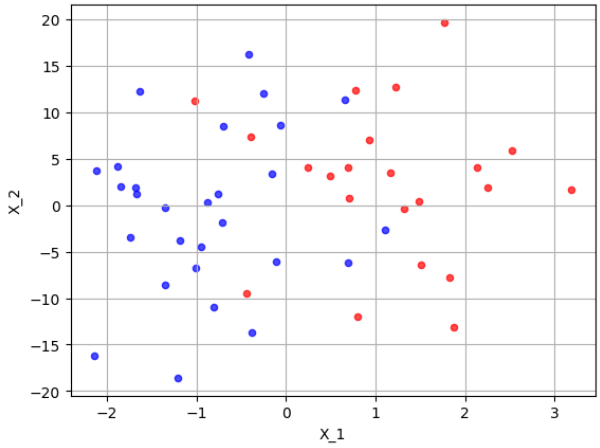

Figure 6: 50 samples of X_1, X_2, Y. The blue points have $Y = 0$, and the red points have $Y = 1$.

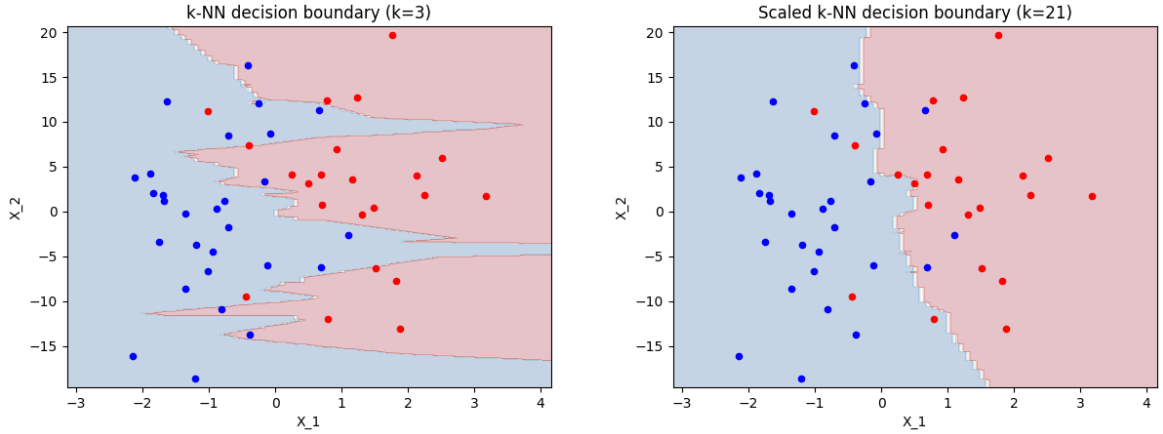

Figure 7: The result of kNN without and with scaling.

Finally, for both classes, $X_2 \sim \mathcal{N}(0, 10)$ and is independent of X_1. We see that only X_1 carries meaningful information (X_2 is completely unrelated to Y), and the best possible classifier for our distribution is

$$Y = 1 \Leftrightarrow X_1 \geq 0.$$

However, because X_2 has a larger scale, it will dominate the distance calculations of kNN.

Without scaling, using 5-fold cross-validation, we find that the optimal value $k = 3$ with the cross-validation error of 0.277. With scaling, we get the best value $k = 9$ with the cross-validation error of 0.172. Fig. 7 depicts the decision boundaries of the best algorithms in both scenarios. Everything has gone wrong in the unscaled case: we have the incorrect value of k, are overfitting, and have significantly higher error than we should. In the scaled case, however, we have learned a reasonable model that is close to the theoretically best possible model mentioned above.

As we see from the images too, when the value of k is small, the decision boundaries are jagged. They become smoother as we increase k. When k is equal to the training data set, then the algorithm always makes the same prediction. In kNN, overfitting happens if we choose k to be too small, and underfitting happens when k is too large. In case of kNN, Occam's razor points us to larger values of k: these are "simpler" than the ones with small k.

8 Decision trees

Decision trees are generic models that can be used for both classification and regression. They generally assume $X_1, \ldots, X_m$ to be numeric, but apart from this restriction, can be applied in any situation. Operationally, a decision tree is basically a (possibly complicated) if-then-else block. Fig. 8 depicts a possible decision tree of depth 2 for classifying iris species. Observe that we ask questions

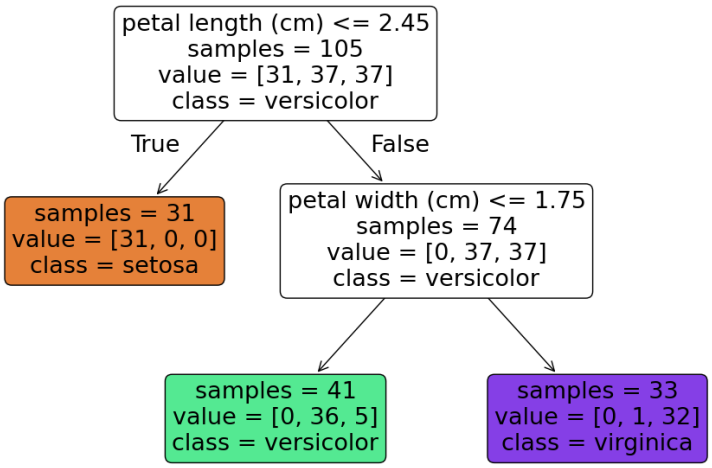

Figure 8: Decision tree for 105 iris samples, where 31 belong to the class "setosa", 37 are "versicolor", and 37 are "virginica". The independent features used are "petal length" and "petal width".

about $X_1, \ldots, X_m$ at the internal nodes, and predict Y at the leaves. What needs to be learned is the order of questions to be asked, and the threshold values (e.g. 2.45 for petal length).

Decision trees, without any restrictions, are extremely prone to overfitting: if we allow arbitrary depth, then the tree will simply perfectly learn the training data. Hence we usually treat the depth of the tree as a hyperparameter, and tune it using cross-validation. Finding the best possible tree (for example, one that maximises accuracy) is computationally hard due to the large number of possible trees and inapplicability of calculus/convex analysis methods. Hence the algorithms for learning trees (e.g. the CART algorithm, which is what `sklearn` uses) try to find a good split at each step, in a greedy fashion. Nevertheless, decision trees tend to perform very well in practice.

A popular and powerful tree-based learning algorithm is *random forest*. It is an *ensemble learning* algorithm, meaning that it trains many "weak learners", which themselves are trees, and aggregates their predictions using a clever weighing strategy.

9 Artificial neural networks

A *neuron* with k in puts is a function of the form $f(a_1x_1 + \cdots + a_kx_k + b)$, where $x_1, \ldots, x_k$ are real-valued inputs, $a_1, \ldots, a_k, b$ are the parameters (i.e., coefficients) of the neuron, and f is a (usually non-linear) activation function (which can be viewed as a hyperparameter) with type $f\colon \mathbb{R} \to \mathbb{R}$. Typical activation functions are the sigmoid σ and

$$\mathrm{ReLU}(x) \coloneqq \begin{cases} 0, & \text{if } x < 0 \\ x, & \text{if } x \geq 0. \end{cases}$$

Logistic regression with binary Y as well as linear regression can be seen learning a single neuron from data. We can link the outputs and inputs of a collection of neurons to build a *neural network*. Let us look at a *fully-connected*, *feed-forward* neural network with a single *hidden layer* with 5 neurons (the choice of 5 was arbitrary; the sizes of hidden layers are hyperparameters that need to be tuned) for classifying iris species as an example.

Recall that the iris data has columns $X_1, \ldots, X_4$ and $Y \in \{0, 1, 2\}$. For neural networks to work well, the input data (i.e., the independent variables) need to be scaled, which we assume to be the case in our example. Fig. 9 depicts our network. Let us assume that the activation functions of all neurons in the hidden layer is f, e.g. ReLU. For $1 \leq i \leq 12$, write z_i for the output of the neuron with number i.

The neurons $1, 2, 3, 4$ are *input neurons*. These take one input and output it as it is. The neuron 5 takes z_1, z_2, z_3, z_4 (which are the values of X_1, X_2, X_3, X_4), and computes

$$z_5 = f(a_1^{(5)}z_1 + a_2^{(5)}z_2 + a_3^{(5)}z_3 + a_4^{(5)}z_4 + b^{(5)})$$

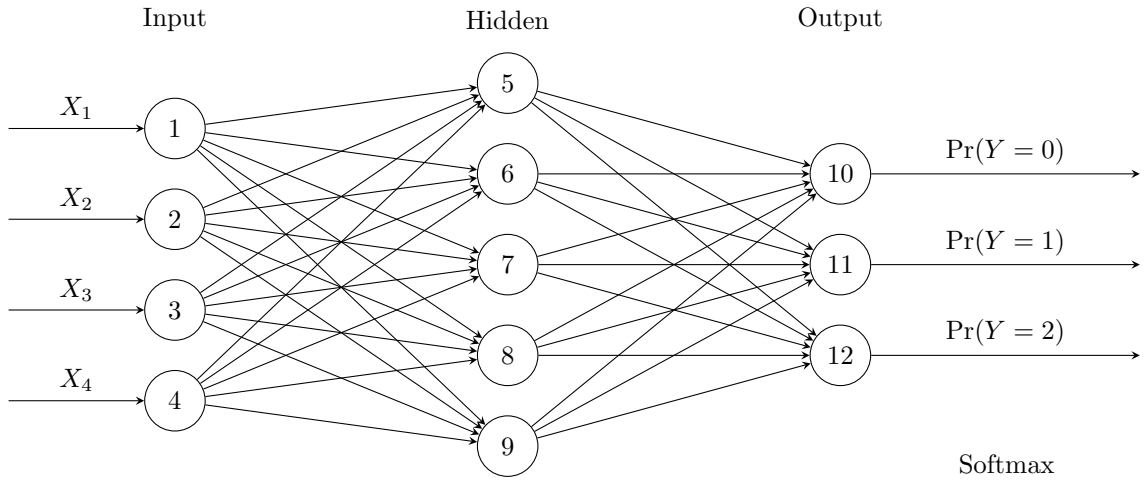

Figure 9: A simple feed-forward neural network.

where $a_1^{(5)}, \ldots, a_4^{(5)}, b^{(5)}$ are its coefficients and f is the activation function. More generally, for $5 \leq i \leq 9$, the ith neuron computes

$$z_i = f(a_1^{(i)} z_1 + a_2^{(i)} z_2 + a_3^{(i)} z_3 + a_4^{(i)} z_4 + b^{(i)}).$$

Each neuron in the hidden layer thus contributes 5 parameters to our model, with 25 in total from the hidden layer.

The neurons in the output layer function slightly differently. In our case, for $10 \leq i \leq 12$, the ith neuron take $z_5, \ldots, z_9$, and computes

$$a_1^{(i)} z_5 + a_2^{(i)} z_6 + a_3^{(i)} z_7 + a_4^{(i)} z_8 + a_5^{(i)} z_9 + b^{(i)}$$

without applying an activation function; here $a_1^{(i)}, \ldots, a_5^{(i)}, b^{(i)}$ are the coefficients of the ith neuron. At the end, a Softmax activation function is applied to z_{10}, z_{11}, z_{12} together, to produce the three probabilities

$$\frac{e^{z_{10}}}{e^{z_{10}} + e^{z_{11}} + e^{z_{12}}}, \frac{e^{z_{11}}}{e^{z_{10}} + e^{z_{11}} + e^{z_{12}}}, \frac{e^{z_{12}}}{e^{z_{10}} + e^{z_{11}} + e^{z_{12}}}.$$

The output layer contributes $3 \times 6 = 18$ parameters, with the total number of parameters being 43. The hyperparameters of the model, among others, are the number of hidden layers, the number of neurons in each layer, and their activation functions.

How do we train the network? That is, how do find the values for the parameters? In case of classification, as in logistic regression, we (try to) find the parameters that minimise the *cross-entropy loss* on the training data, which is an expression that involves logs, the training data, and the activation functions. In case of regression, the output layer consists of a single neuron without an activation function, and we can use mean squared error as our objective function.

Due to the complex structure of neural networks (in particular, due to the alternating linear/non-linear steps), this objective function in general does not have a unique minimum. Hence we find a *local minimum* using *gradient descent* and starting from a random location in the parameter space. To compute the gradients (i.e., the derivatives) we can use *backpropagation*, which is a method based on the chain rule for calculating derivatives. In practice, full-connected neural networks with one or two layers perform extremely well (often with $0.95 - 0.99$ accuracy on classification tasks) on real-world tabular datasets. With a relatively few number of neurons, these networks can learn any function, i.e. any kind of a relationship between $X_1, \ldots, X_m$ and Y.

Other important neural network architectures include *convolutional neural networks* (for analysing images, audio, and other "spatially related" forms of data), recurrent neural networks (for analysing sequential data like texts and time series), transformers (ChatGPT etc.), graph neural networks (for data represented as nodes and edges), autoencoders (for learning efficient representations of complex data), and so on.

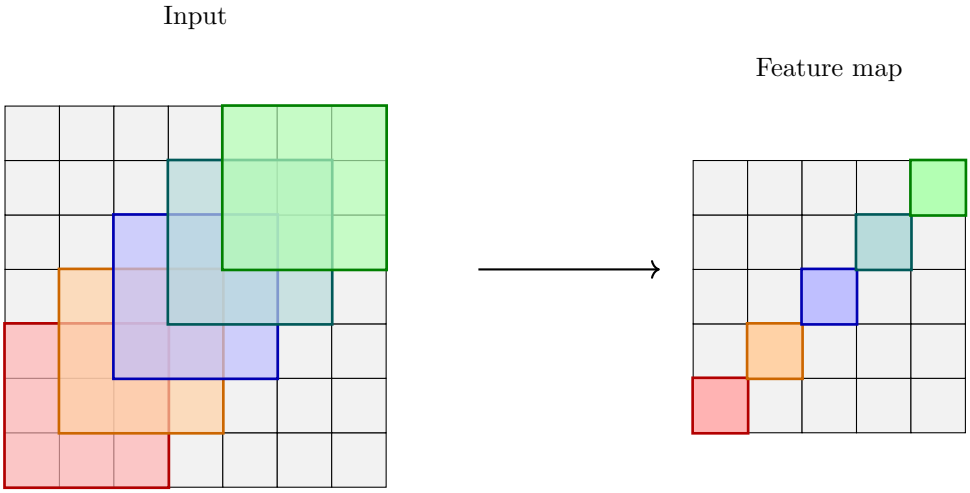

Figure 10: A single 3×3 filter on a 7×7 image.

9.1 Convolutional neural networks

In a *convolutional* neural network (CNN), we have convolution layers where each neuron only sees a window of neighbouring outputs from the previous layer rather than all outputs of the previous layer, which was the case for fully connected (also known as dense) layers.

Suppose we are working with RGB images of size 7×7. We represent such an image as a *tensor* of shape $7 \times 7 \times 3$: think of these as a 3D arrays, with the numbers representing height $\times$ width $\times$ depth. Suppose our first layer is a convolution layer with, say, 10 *filters* of size 3×3. A single filter is just a sliding a window. In our case, a filter is represented by a tensor of shape $3 \times 3 \times 3$. We slide our filter across the image, at each step taking entrywise product with the corresponding entries in the picture, summing the results, and applying an activation function. Each filter produces a new "image" (called a *feature map*) of size $5 \times 5 \times 1$. Therefore, the output of the whole convolutional layer is a tensor of shape $5 \times 5 \times 10$. The idea is that different filters will learn too look for different things in their input tensor. In the setting described above, we have the *padding* of 0 and the *stride* of 1; these describe how far inwards from the edges of the input tensor we begin and at each step by how many squares we slide the window.

Apart from convolutional layers, CNNs use, among others, *pooling*, *global average pooling*, and *flattening* layers. Pooling reduces the size of the tensor by moving a $k \times k$ window with stride > 1, and taking the average or the maximum of what the filter sees at each step. Global average pooling computes the average of whole feature maps. Finally, flattening transforms a 3D tensor into a vector.

We can try to design our own CNNs using cross-validation and trial-and-error. However, it is much more common and efficient to use an already established architecture. We give two examples of architectures from the ImageNet competition that classify $224 \times 224 \times 3$ images into 1000 classes. *AlexNet* has 5 convolutional steps followed by 3 fully connected layers. The first convolution, for example, involves 96 different filters (with ReLU activation) of size 11×11, stride 4 and padding 0; this is followed by max pooling with window size 3×3 and stride 2.[1] *VGG*, on the other hand, is 16 (or 19) layers deep, but uses much smaller filters, e.g. of size 1×1, 3×3, and 5×5.

10 The probabilistic perspective

Everything we have seen so far has revolved around minimising an appropriate loss function. There is an alternative way to develop and justify the same machine learning algorithms without mentioning loss functions, working with probability theory (especially Bayesian inference) instead. (This is also how early modern statisticians like Fischer operated.) Here we very briefly scratch the surface of this rich area.

[1] This is a simplified version: the original AlexNet also contains a *local response normalization* sub-step at the end of the first two convolution steps.

10.1 Maximum likelihood estimate

Suppose we want to fit a model to our data (e.g. a normal distribution), but don't know which parameters we should choose (e.g. the mean and standard deviation). The paradigm of *maximum likelihood estimate* (MLE) states that we should choose the parameter values that maximise the *likelihood* of observing our data. For discrete random variables, likelihood is the same as probability. Let us consider few examples.

10.2 Bernoulli distribution

Suppose we have a biased coin that comes up with heads with probability $0 < p < 1$. We toss it 10 times and observe 6 heads and 4 tails. (Of course, the argument applies to any number of observed heads.) Which hypothesis regarding the value of p should we choose?

Recall that the number of heads observed follows the binomial distribution with parameters $n = 10$ and p. Following the MLE paradigm, we write the probability of observing our data for each value of p as $\ell(p) = \binom{10}{6} p^6(1-p)^4$. We want to find p that maximises this quantity. To this end, we take the derivative with respect to p to obtain

$$\ell'(p) = \binom{10}{6}(6p^5(1-p)^4 - 4(1-p)^3p^6) = \binom{10}{6}p^5(1-p)^3(6-10p).$$

Setting the derivative to zero, we obtain that g has a single critical point at $p = \frac{6}{10}$, which is the intuitive value for p that anyone would predict given our observation of 6 heads and 4 tails. To formally argue that $p = \frac{6}{10}$ is a global maximum of g we need to (1) take the second derivative and show that its value is negative at the critical point, and (2) show that $\lim_{p\to 0} \ell(p)$ and $\lim_{p\to 1} \ell(p)$ are not greater than the value at the critical point; in our case both limits are in fact zero.

10.3 Univariate normal distribution

Next, let us consider the case of a univariate normal distribution. Suppose we are given observations $x_1, \ldots, x_m \in \mathbb{R}$, and want to find the normal distribution that best fits our data. For continuous random variables, comparing the probabilities of observing m particular points does not make sense: such probabilities are always zero. Instead, we use the *likelihood*. For $x \in \mathbb{R}$, the likelihood of observing x in the model given by μ, σ^2 is defined as

$$\mathcal{L}(x; \mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{1}{2\sigma^2}(x-\mu)^2}$$

where the right-hand side is the probability density function of the normal distribution with mean μ and variance σ^2 evaluated at the point x. Observe that the likelihood is always positive but should not be interpreted as a probability: in particular, it can be greater than 1. The likelihood of observing $x_1, \ldots, x_m$ is then simply defined as

$$\mathcal{L}(x_1, \ldots, x_m; \mu, \sigma^2) = \prod_{i=1}^{m} \mathcal{L}(x_i; \mu, \sigma^2) = \left(\frac{1}{\sqrt{2\pi\sigma^2}}\right)^m e^{-\frac{1}{2\sigma^2}\sum_{i=1}^{m}(x_i-\mu)^2}.$$

This definition is justified by the fact that $x_1, \ldots, x_m$ are assumed to be independent: recall that $P(A \text{ and } B) = P(A)P(B)$ for independent events A, B. Hence to perform an MLE estimate on our data $x_1, \ldots, x_m$ we have to solve the following optimisation problem. Find $\mu \in \mathbb{R}$ and $\sigma^2 > 0$ that maximises the value of $\mathcal{L}(x_1, \ldots, x_m; \mu, \sigma^2)$. We next show how to do this. We additionally assume that it is not the case that $x_1 = \cdots = x_m$, because otherwise it is not possible to conclude anything from the data about the variance. Mathematically, without this assumption we get that $\mathcal{L}(x_1, \ldots, x_m; \mu, \sigma^2)$ does not have global minimum because we can choose $\mu = x_1$ and make $\mathcal{L}(x_1, \ldots, x_m; \mu, \sigma^2)$ arbitrarily small by taking $\sigma^2 \to 0$.

Because the natural logarithm $\log\colon (0, \infty) \to \mathbb{R}$ is a strictly increasing function (i.e. $\log(x) < \log(y)$

if and only if $x < y$), our problem is equivalent to maximising the log-likelihood

$$\begin{aligned}\ell(x_1, \ldots, x_m; \mu, \sigma^2) &= \log(\mathcal{L}(x_1, \ldots, x_m; \mu, \sigma^2)) \\ &= m \log\left(\frac{1}{\sqrt{2\pi\sigma^2}}\right) - \frac{1}{2\sigma^2}\sum_{i=1}^{m}(x_i - \mu)^2 \\ &= -\frac{m}{2}\log(2\pi\sigma^2) - \frac{1}{2\sigma^2}\sum_{i=1}^{m}(x_i - \mu)^2\end{aligned}$$

which we simply denote by ℓ. Note that in the expression above, every occurrence of σ is squared. Hence we can think in terms of variance and not standard deviation. We now apply calculus: every local maximum must satisfy

$$\begin{aligned}\frac{\partial \ell}{\partial \mu} &= \frac{1}{\sigma^2}\sum_{i=1}^{m}(x_i - \mu) = 0 \\ \frac{\partial \ell}{\partial \sigma^2} &= -\frac{m}{2\sigma^2} + \frac{1}{2\sigma^4}\sum_{i=1}^{m}(x_i - \mu)^2 = 0\end{aligned}$$

where we have applied the chain rule for differentiation. From the first equation,

$$\frac{1}{\sigma^2}\sum_{i=1}^{m}(x_i - \mu) = 0 \;\Leftrightarrow\; \sum_{i=1}^{m}(x_i - \mu) = 0 \;\Leftrightarrow\; \sum_{i=1}^{m} x_i = m\mu \;\Leftrightarrow\; \mu = \frac{1}{m}\sum_{i=1}^{m} x_i.$$

That is, at all critical points of ℓ (which as a function of μ and σ^2) the value of μ is the *sample mean*, which is natural and intuitive. On the other hand,

$$-\frac{m}{2\sigma^2} + \frac{1}{2\sigma^4}\sum_{i=1}^{m}(x_i - \mu)^2 = 0 \;\Leftrightarrow\; \sum_{i=1}^{m}(x_i - \mu)^2 = m\sigma^2 \;\Leftrightarrow\; \sigma^2 = \frac{1}{m}\sum_{i=1}^{m}(x_i - \mu)^2.$$

That is, at every critical point σ^2 is equal to the *sample variance*. This is still intuitive, but not as unequivocal as the critical point we obtained for μ: a different but well-known estimate of the population variance σ^2 given $x_1, \ldots, x_m$ is the *unbiased sample variance* $\frac{1}{m-1}\sum_{i=1}^{m}(x_i - \mu)^2$, which is often preferred in practice.

So far we have shown that ℓ has a single critical point given by

$$\mu = \frac{1}{m}\sum_{i=1}^{m} x_i \tag{1}$$

and

$$\sigma^2 = \frac{1}{m}\sum_{i=1}^{m}(x_i - \mu)^2. \tag{2}$$

It remains to determine the nature of this point. To this end, we use the *second partial derivative test*. We have that

$$\begin{aligned}\frac{\partial^2 \ell}{\partial \mu\, \partial \sigma^2} &= \frac{\partial^2 \ell}{\partial \mu\, \partial \sigma^2} = -\frac{1}{\sigma^4}\sum_{i=1}^{m}(x_i - \mu) \\ \frac{\partial^2 \ell}{\partial \mu^2} &= -\frac{m}{\sigma^2} \\ \frac{\partial^2 \ell}{\partial (\sigma^2)^2} &= \frac{m}{2\sigma^4} - \frac{1}{\sigma^6}\sum_{i=1}^{m}(x_i - \mu)^2\end{aligned}$$

where we have again used the chain rule. At our critical point, the value of $\frac{\partial^2 \ell}{\partial \mu^2}$ is negative, and the quantity

$$D(\mu, \sigma^2) = \frac{\partial^2 \ell}{\partial \mu^2} \cdot \frac{\partial^2 \ell}{\partial (\sigma^2)^2} - \frac{\partial^2 \ell}{\partial \mu\, \partial \sigma^2} \cdot \frac{\partial^2 \ell}{\partial \sigma^2\, \partial \mu}$$

is equal to $\frac{m^2}{2\sigma^6}$, which is positive. By the second partial derivative test, we conclude that (μ, σ^2) defined by the equations (1) and (2) is a local maximum. Finally, observe that for any fixed μ, we have that $\ell \to -\infty$ as $\sigma^2 \to \infty$. Hence our unique critical point is the (unique) global maximum.

10.4 Maximum a posteriori estimate

Suppose we have fit a probabilistic model to our data, e.g. using MLE. More concretely, suppose we have got the random variables $X_1, \ldots, X_m, Y$ (where we want to predict Y given $X_1, \ldots, X_m$), and have determined the joint probability distribution function $f_y(x_1, \ldots, x_m)$ for every possible value of y. Given $X_1 = x_1, \ldots, X_m = x_m$, we can simply predict the value of Y to be y such that the likelihood of observing $X_1 = x_1, \ldots, X_m = x_m$ assuming $Y = y$, i.e. $f_y(x_1, \ldots, x_m)$, is maximised. But what if some values of y are very rare? To maximise accuracy, we want to somehow predict such y less frequently that other, more common values of Y. The solution is the maximum a posterior estimate (MAP), which is derived from *Bayes' theorem*. We have that

$$\mathrm{P}(Y = y \mid X_1 = x_1, \ldots, X_m = x_m) = \frac{f_y(x_1, \ldots, x_m) \cdot \mathrm{P}(Y = y)}{f(x_1, \ldots, x_m)}.$$

Here $f(x_1, \ldots, x_m)$ is the joint distribution function of $X_1, \ldots, X_m$, but note that it does not depend on y. Hence we choose y to maximise $f_y(x_1, \ldots, x_m) \cdot \mathrm{P}(Y = y)$. Note that we need the value of $\mathrm{P}(Y = y)$: the latter can be estimated as the proportion of points with $Y = y$ in our dataset.

A Some Python code

A.1 Generating data for Sec. 3

```
import numpy as np

def f(x):
        return 0.105*(x**3)-1.68*(x**2)+7.86*x-5.29

def sample():
        x = np.random.uniform(1, 10)
        y = f(x) + np.random.normal(0, 1)
return x, y

n = 50
X = []
Y = []

np.random.seed(0)

for _ in range(n):
        x, y = sample()
        X.append(x)
        Y.append(y)
```

A.2 Generating data for Sec. 7

```
import numpy as np

def generate_data(n, random_state):
        np.random.seed(random_state)

        # Compute the array of y's
        Y = np.random.choice([0, 1], size=n, p=[0.5, 0.5])

        # Initialize X
        X = np.zeros((n, 2))

        # Generate the first column of X based on Y
        X[Y == 0, 0] = np.random.normal(loc=-1, scale=1, size=np.sum(Y == 0))
```

```
        X[Y == 1, 0] = np.random.normal(loc=1, scale=1, size=np.sum(Y == 1))

        # Generate the second column of X
        X[:, 1] = np.random.normal(loc=0, scale=10, size=n)

return X, Y

n = 50
X, Y = generate_data(n, 1)
```